ATM TRANSACTION SEQUENCE SIMULATION

Assignment Matter • Algorithm • Pseudocode • Sample Input and Output

1. Problem Statement

Develop a Python program that simulates a sequence of ATM deposits and withdrawals. The program starts with an initial account balance, processes a specified number of transactions, validates each transaction, enforces a maximum withdrawal limit of Rs. 20,000, prevents withdrawals that exceed the available balance, and displays a summary of successful deposits and withdrawals along with the final balance.

2. Rules and Conditions

Item	Condition / Action
Initial balance	Read the starting balance from the user.
Transaction count	Read how many transactions must be processed.
Deposit (D)	Add a positive amount to the balance and update deposit count and total.
Withdrawal (W)	Allow only if the amount is within Rs. 20,000 and does not exceed the current balance.
Invalid amount	If the amount is zero or negative, reject the transaction.
Invalid type	Accept only D for deposit or W for withdrawal; reject any other input.
Summary	Display successful deposit/withdrawal counts, their totals, and the final balance.

3. Algorithm

1. Start the program.

2. Read the initial account balance and the number of transactions.

3. Set the withdrawal limit to Rs. 20,000 and initialize deposit/withdrawal counters and totals to zero.

4. Repeat the following steps for each transaction.

5. Read the transaction type and amount.

6. If the amount is zero or negative, reject it and move to the next transaction.

7. If the type is D, add the amount to the balance and update the deposit count and total.

8. If the type is W, check the withdrawal limit and whether sufficient balance is available. If both checks pass, subtract the amount and update the withdrawal count and total.

9. If the transaction type is neither D nor W, reject it.

10. After all transactions, display the transaction summary and final balance.

11. Stop the program.

4. Pseudocode

```
BEGIN
INPUT balance
INPUT num_transactions
SET WITHDRAWAL_LIMIT = 20000
SET deposit_count = 0, withdraw_count = 0
SET deposit_total = 0, withdraw_total = 0
FOR each transaction
  INPUT transaction_type
  INPUT amount
  IF amount <= 0
    DISPLAY "Invalid amount; transaction rejected"
    CONTINUE
  END IF
  IF transaction_type = "D"
    balance = balance + amount
    deposit_count = deposit_count + 1
    deposit_total = deposit_total + amount
  ELSE IF transaction_type = "W"
    IF amount > WITHDRAWAL_LIMIT
      DISPLAY "Withdrawal exceeds limit"
    ELSE IF amount > balance
      DISPLAY "Insufficient balance"
    ELSE
      balance = balance - amount
      withdraw_count = withdraw_count + 1
      withdraw_total = withdraw_total + amount
    END IF
  ELSE
    DISPLAY "Invalid transaction type"
  END IF
END FOR
DISPLAY deposit_count, deposit_total
DISPLAY withdraw_count, withdraw_total
DISPLAY balance
END
```

5. Sample Input and Output

Enter initial balance: 10000
Enter number of transactions: 5

Transaction 1 — Type: D, Amount: 5000 — Deposit successful.
Transaction 2 — Type: W, Amount: 3000 — Withdrawal successful.
Transaction 3 — Type: W, Amount: 25000 — Withdrawal exceeds limit; rejected.
Transaction 4 — Type: W, Amount: 15000 — Insufficient balance; rejected.
Transaction 5 — Type: D, Amount: 2000 — Deposit successful.

ATM TRANSACTION SUMMARY
Total Deposits: 2
Deposit Amount: Rs. 7000.00
Total Withdrawals: 1
Withdrawal Amount: Rs. 3000.00
Final Balance: Rs. 14000.00

6. Concepts Used

Variables: balance, num_transactions, amount, transaction_type, deposit_count, and withdraw_count. Data types: float, integer, and string. Operators: arithmetic (+, -), comparison (<=, >, ==), and assignment (+=). Conditional statements: if, elif, and else. Loop: for loop to process transactions. String operation: upper() converts the transaction type to uppercase. Control statement: continue skips the rest of the current loop iteration for an invalid amount.

7. Conclusion

The ATM simulation processes deposits and withdrawals while checking transaction validity, enforcing the withdrawal limit, and preventing overdrafts. It maintains the account balance and produces a clear summary of successful transactions. The program demonstrates variables, data types, operators, loops, conditional statements, strings, and control statements in Python.

Note: This is an educational simulation. It does not connect to a real bank account or ATM.